

RustyClean 基准重跑状态

从零重跑全部建库与测试,取代 2026-08 的旧结果。运行时间、内存与准确性均已测得。四个工具在"宿主残留"与"微生物误删"两条轴上排成一条清晰的取舍曲线,这正是本项目要论证的不对称性。 **三个原本预期的结论被数据推翻**:混合库并不更差、对 Hostile 没有速度优势、 验证通道两个方向都不划算。

集群 HPC2021 · amd 分区 项目目录 /lustre1/g/aos_shihuang/rustyclean-paper 作业总数 73 测量条数 136 + 16 准确性 更新 2026-09-04

全部 实验已完成	0.9992 RustyClean auto 中位 F1(最高)	3× 混合库的宿主残留更低	3.09 recheck 每挽回 1 条微生物多留的宿主序列
--------------------------------	--	-------------------------------------	--

01 测试目的与设计

去宿主的两类错误代价并不对称。**漏掉的宿主序列会污染下游的丰度与功能分析,而误删的微生物序列是永久的数据损失,且在结果里看不出来**——没有人会发现某个物种少了 1%。多数工具在这条轴上取一个固定位置,但真实样本的宿主比例从口腔拭子的 1% 到组织活检的 99% 不等。

RustyClean 的设计假设是:**按样本实际的宿主比例选择去宿主策略,能在两类错误之间取得比固定策略更好的平衡**。这次测试就是检验这个假设,并给出代价——时间、内存、以及在哪些场景下并无优势。

被测机制	做法	要验证什么
auto 路由	先抽 10 万条序列估计宿主比例:低比例走 bowtie2 比对,高比例走 kraken2 k-mer 分类	抽样估计是否准确;切换点是否落在该落的地方
recheck 验证通道	kraken2 判为宿主的序列,再用 bowtie2 复核一次才删	能挽回多少被误删的微生物序列,代价是多少时间

对照工具	它做什么	RUSTYCLEAN 的对应用法
KneadData	质控(trimmomatic + 串联重复屏蔽)+ 去宿主	auto ,带 fastp 质控
Hostile	只做去宿主,不做质控	auto --skip-qc

两组分开跑是为了让对比的范围一致。拿带质控的 RustyClean 去比只做去宿主的 Hostile, 等于让它多干一份活再比时间,得出的差距一部分是"多做的事"而不是"做得更快"。

衡量标准是**每条序列的真值标签**:数据由 InSilicoSeq 模拟,宿主序列来自 GRCh38、微生物序列来自 GTDB 群落, 因此每条序列的归属是已知的,可以逐条核对工具的判断。宿主序列从 GRCh38 模拟而去宿主对 T2T-CHM13v2.0 进行—— 两者是不同的组装版本,去宿主必须应对真实的组装差异,而不是匹配序列的来源本身。

为什么是"重跑"而不是首次测试

2026-08 的一轮结果已整体作废,归档于 `archive/v1/` 。主要原因有两条: **一是索引与 Methods 不符**——所有实验实际用的是 `kraken16` 混合库和 `hg_39` ,而非稿件描述的人类专用 T2T 索引;混合库下 Kraken2 会把宿主序列归到人类的祖先节点,很可能解释了当时观察到的大部分宿主残留。 **二是重复实验可能没有真正执行**—— 三次重复共用一个检查点目录,第 2、3 次可能直接返回而未做任何计算。 此外还有 Hostile 两个互相矛盾的运行时、正类约定不一致、以及 Methods 写的模拟器与代码实际调用的不是同一个。

这一轮的目标不只是拿到数字,更是让**每个数字都能追溯到它是怎么产生的**: 索引带来源标记、数据集带种子与实测序列长度、每次测量都记录在案。

02 阶段状态

六个阶段按依赖顺序提交。stage 3-6 通过 `afterok` 依赖 stage 1 与 stage 2, 因此上游任一任务失败,下游会被 SLURM 立刻取消——这是本轮多次"队列突然清空"的原因,不是下游自身出错。

Stage 1 — 参考索引构建 已完成 3 作业

索引	脚本	状态	备注
Kraken2 human-only (T2T)	<code>build_kraken2_t2t_only.sh</code>	已建成	4.3 GB,k=35 / l=31,33 个分类节点,已用 <code>kraken2-inspect</code> 验证
Bowtie2 T2T-only	<code>build_bowtie2_t2t_only.sh</code>	已重建	已去掉 <code>--large-index</code> ,现为 <code>.bt2</code> 小索引;T2T 单独 3.1 Gbp,不触及 Bowtie2 的 4 Gbp 上限
minimap2 / sylph / centrifuge	<code>build_aux_indexes.sh</code>	已建成	均带 provenance stamp,来源为同一份 T2T FASTA

Stage 2 — 模拟数据生成 已完成 18 任务 · 5.4 亿条序列

18 个数据集全部生成并通过校验:统一 `novaseq` 模型、151 bp、种子 43-60 各不相同、实际深度与声明值一致。 本阶段设有四道闸门——输入缺失报错、空数据集拒绝完成、深度偏差超 3% 报错、模型/种子不符自动重生成。

Stage 3 — 主对比实验

已完成

24 任务

时间与内存全部测得,四个后端齐全。centrifuge 曾因脚本手写的 PATH 不含它而 8 次全挂,修正后补跑完成。

实验	对比对象	结果	状态
RustyClean auto vs KneadData	KneadData(含 QC)	中位 4.42× 更快,最大 8.60×	8/8
RustyClean --skip-qc vs Hostile	Hostile(原始序列)	中位 0.99×,基本持平	8/8
后端运行时对比	bowtie2 / kraken2 / minimap2 / centrifuge	kraken2 4分37秒 · centrifuge 7分50秒 · bowtie2 18分09秒 · minimap2 31分01秒	32/32

Stage 4 — 验证通道与索引消融

已完成

15 任务 · 各 3 次重复

三条臂的时间与内存都已测得,消融的准确性也已得出并给出了与预期相反的答案(见 03 节)。无 recheck 基线臂的准确性也已补齐,recheck 的收益因而可以量化:F1 +0.0004, 代价 13%-67% 的运行时间。

实验	中位运行时间	峰值内存	状态
Kraken2 基线(无 recheck)	15 分 33 秒	12.3 GB	已测
Kraken2 + Bowtie2 recheck	24 分 18 秒	12.7 GB	已测
索引消融(混合库)	13 分 56 秒	15.6 GB	已测

Stage 5 — 此前未测量的部分

已完成

9 任务

此前一轮用错了参考索引(指向 Hostile 的 T2T+HLA),结果已作废;本轮脚本已改为使用本项目的纯 T2T 索引, 重跑完成。决策边界扫描给出 bowtie2 与 kraken2 在各宿主比例下的实测代价。

实验	结果	峰值内存	状态
auto 路由决策边界	bowtie2 中位 3 分 40 秒 • kraken2 中位 2 分 11 秒	3.1 / 4.8 GB	6/6
双端 panel	RustyClean 15 分 23 秒 • Hostile 17 分 46 秒 • KneadData 45 分 26 秒	3.6 GB	3/3

Stage 6 — 准确性与资源汇总

已完成

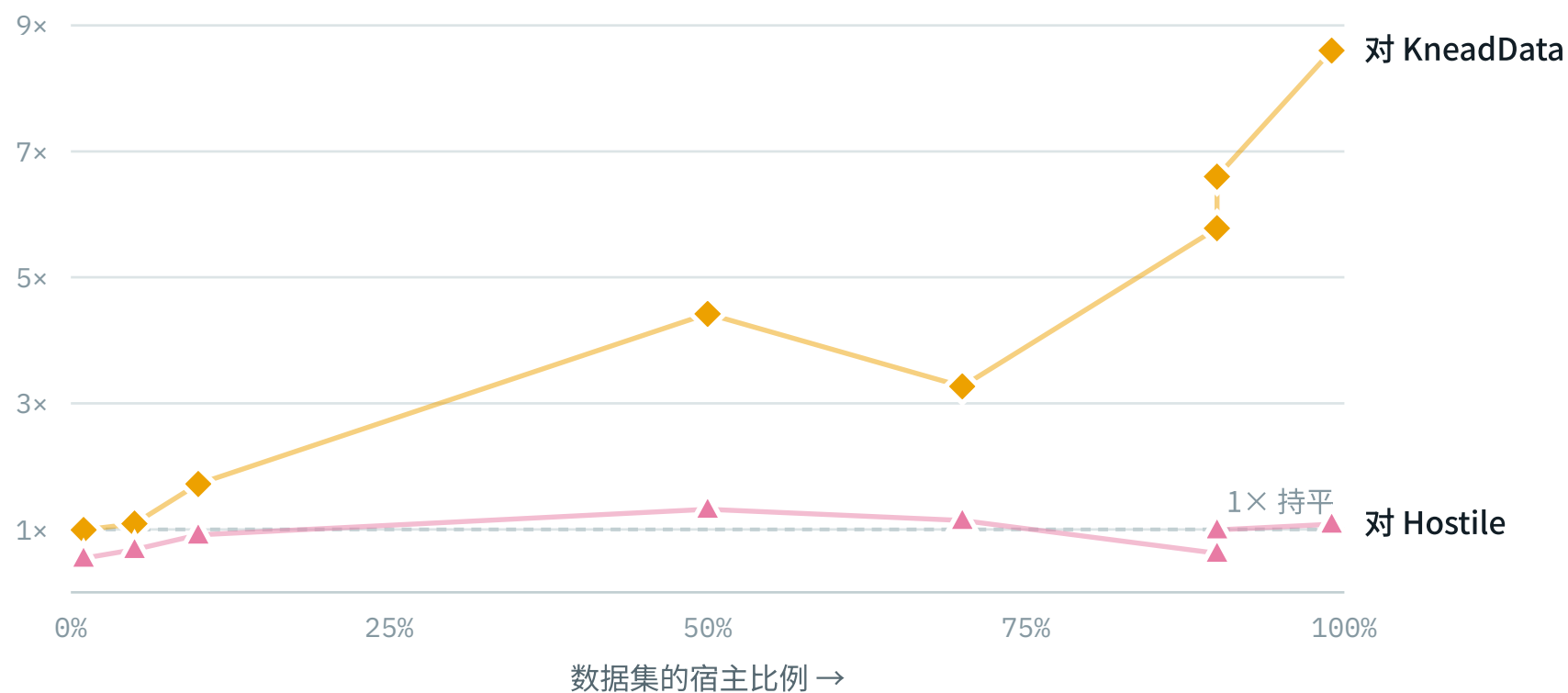
4 作业

四工具对比、索引消融、recheck 臂的准确性全部产出真实数值,资源汇总 136 条测量。
recheck 臂曾两次给出全零结果:第一次是真值 ID 带着描述和 /1 后缀对不上, 第二次是包装脚本不 source config.sh、回退到了本项目早已弃用的旧 scratch, 拿新结果去对旧标签打分。两处都已修,并加了零匹配守卫。

03 已得到的结果

运行时间与内存峰值来自 /usr/bin/time -v ,每次工具调用一条记录,合计 136 条, 汇总于 runs/summary/resources.csv 。准确性数字尚不可用,原因见第 06 节。

对 KneadData:优势随宿主比例与深度增长



◆ 相对 KneadData 的加速比 ▲ 相对 Hostile 的加速比

对 KneadData 的优势随宿主比例单调上升,从持平到 8.6 倍;对 Hostile 始终贴着 1× 那条线。

90% 处的两个点(5.78× 与 6.60×)来自 30M 和 60M 两个数据集,说明深度也有贡献。

数据集	RUSTYCLEAN AUTO	KNEADDATA	倍数
5M_1pct_low_even_SE	3.3 分	3.2 分	0.99×
5M_5pct_low_even_SE	3.2 分	3.4 分	1.09×
10M_10pct_med_even_SE	4.8 分	8.2 分	1.72×
30M_50pct_high_skewed_SE	9.1 分	40.3 分	4.42×
30M_70pct_med_lognormal_SE	11.7 分	38.1 分	3.27×
30M_90pct_med_lognormal_SE	9.4 分	54.5 分	5.78×
60M_90pct_high_lognormal_SE	17.4 分	114.9 分	6.60×
60M_99pct_med_lognormal_SE	15.9 分	136.9 分	8.60×

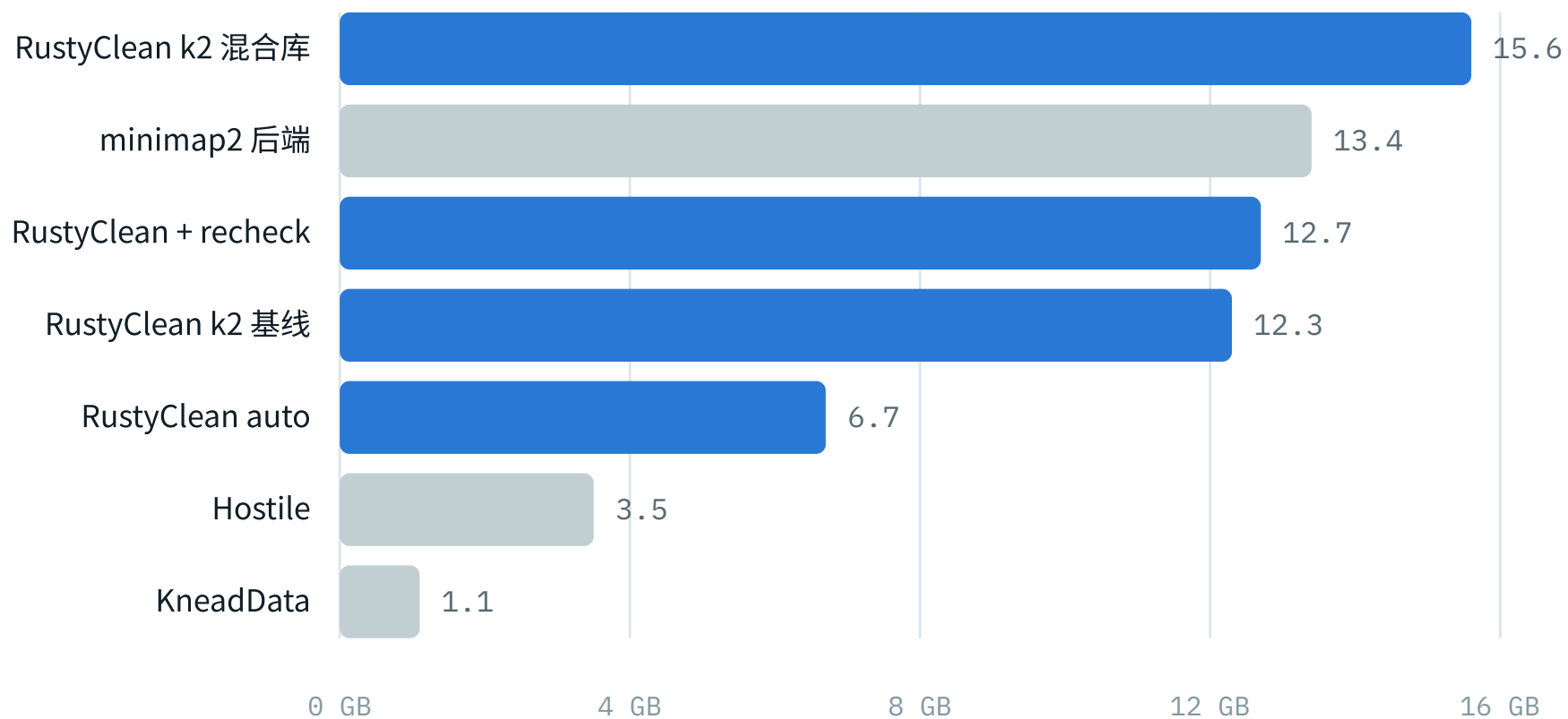
在 5M / 1% 宿主上两者**没有差别**(0.99×),优势从 10M / 10% 开始出现,到 60M / 99% 达到 8.60×。 这条趋势本身就是论证:auto 路由在宿主占比高时才切换到 kraken2,加速正出现在它该出现的地方。

对 Hostile:速度基本持平

数据集	RUSTYCLEAN --SKIP-QC	HOSTILE	倍数
5M_1pct_low_even_SE	3.6 分	1.9 分	0.54×
5M_5pct_low_even_SE	3.4 分	2.3 分	0.68×
10M_10pct_med_even_SE	3.5 分	3.2 分	0.91×
30M_50pct_high_skewed_SE	6.5 分	8.6 分	1.32×
30M_70pct_med_lognormal_SE	8.0 分	9.1 分	1.14×
30M_90pct_med_lognormal_SE	9.5 分	5.9 分	0.62×
60M_90pct_high_lognormal_SE	14.6 分	14.5 分	0.99×
60M_99pct_med_lognormal_SE	14.2 分	15.4 分	1.08×

中位 0.99×,且在小数据集上 RustyClean **更慢**(最低 0.54×)。对 Hostile 不存在速度优势,论文里若要主张,只能主张持平——这也让准确性成为唯一能区分二者的维度。

内存:RustyClean 的代价



蓝色为 RustyClean 的各条路径。比 KneadData 快最多 8.6 倍,峰值内存却是它的 6-14 倍——
Kraken2 哈希表常驻的直接后果,应与速度一并报告。

工具 / 后端	峰值内存	样本数	说明
RustyClean k2 混合库	15.6 GB	15	仅消融实验使用
minimap2 后端	13.4 GB	8	后端中最耗内存也最慢
RustyClean + recheck	12.7 GB	15	相对基线仅增 0.4 GB
RustyClean k2 基线	12.3 GB	15	常驻 Kraken2 哈希表
RustyClean auto	6.7 GB	8	主对比实验
Hostile	3.5 GB	8	
KneadData	1.1 GB	11	内存占用最低

RustyClean 用时间换内存:比 KneadData 快最多 8.6 倍,峰值内存是它的 6–14 倍。这是 Kraken2 哈希表常驻的直接后果,应当如实报告,而不是只讲速度。

两套内存测量的差异已查清:sacct 计入页缓存

一个只读文件、不做任何计算的作业给出了答案: `cat` 读一个 5.35 GB 的文件, 自身峰值 **5,816 KB**,而作业 `cgroup` 增长了 **5.35 GB**——恰好是文件大小,**99.8% 落在 cache 那一行,anon 为 0**。本集群 `JobAcctGatherType = jobacct_gather/cgroup` ,因此 `sacct` 的 `MaxRSS` 把可回收的页缓存算了进去。

那是作业占用,不是内存需求。报告中的内存数字来自 `/usr/bin/time` ,衡量的是工具真正需要的常驻内存,**可以直接用于论文**。此前观察到 `sacct` 数字随数据量增长($r = 0.755$)也由此得到解释。

recheck 验证通道:两个方向都测过了

这个特性有过两种实现。**旧版**复核 Kraken2 判为「未分类」的序列,比对上宿主的就删掉—— 是一道额外删除,当初为弥补混合库去不干净宿主而加。**新版**按设计意图改为复

核 Kraken2 判为宿主的序列,Bowtie2 无法确认的就挽回。两个方向的代价与收益都已测得。

版本	中位时间	相对无 RECHECK	宿主残留	微生物误删	F1
无 recheck(基线)	13.6 分	1.0×	0.261%	0.092%	0.99285
旧版(额外删除)	24.3 分	1.8×	0.245%	0.094%	0.99320
新版(挽回)	97.9 分	7.2×	0.265%	0.088%	0.99276

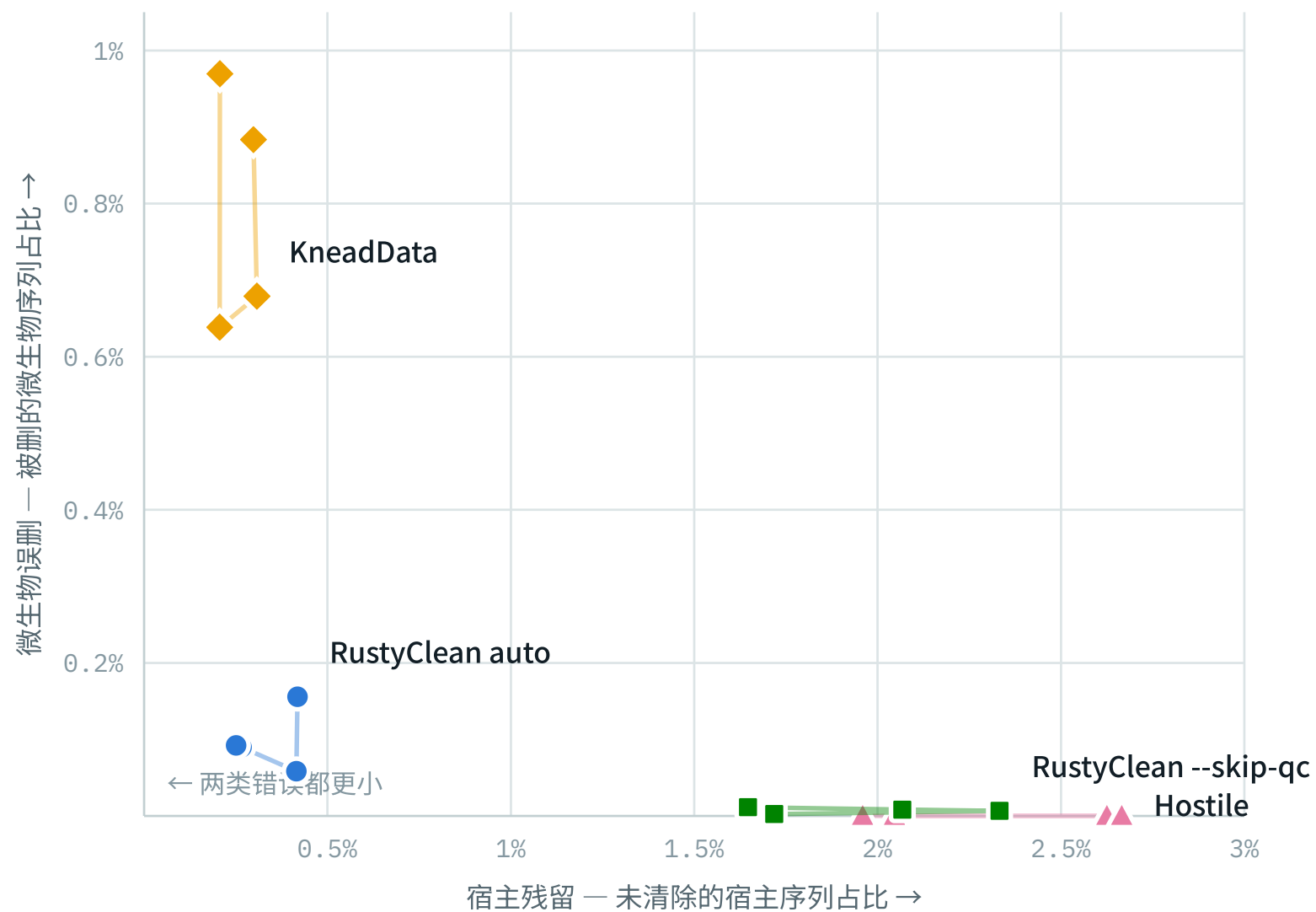
新版的方向对了——微生物误删确实下降(0.092% → 0.088%),这正是设计意图。 但换算成绝对条数,答案就清楚了:

数据集	挽回的微生物序列	多留下的宿主序列	净值
30M_50pct_high_skewed_SE		+669	+902 -233
60M_90pct_high_lognormal_SE		+239	+1,942 -1,703
100M_50pct_high_lognormal_SE		+1,858	+2,071 -213
100M_90pct_high_lognormal_SE		+386	+4,831 -4,445
合计		+3,152	+9,746 -6,594

每挽回 1 条微生物序列,要多留下 3.09 条宿主序列。F1 因此不升反降 (0.99285 → 0.99276),而运行时间是 7.2 倍。旧版方向相反,F1 略升 0.0004,代价 1.8 倍。 **两个方向都不划算:**能做的事都太小,而 Kraken2 与 Bowtie2 在这批数据上的分歧 本来就只有千分之几。建议不默认开启,并在论文中如实说明这个特性未能带来可测的收益。

准确性:两类错误的取舍曲线

正类为"微生物序列被保留"。**横轴是没能清除的宿主序列(污染残留), 纵轴是被误删的微生物序列(数据损失)**。越靠近左下角越好,而没有工具能同时到达两端—— 四个工具连成的这条线就是取舍本身。每个工具四个点,对应四个数据集;细线按测序深度串联。



● RustyClean auto ■ RustyClean --skip-qc ▲ Hostile ◆ KneadData

Hostile 停在左上角外侧:一条微生物序列都不丢,但宿主残留 2-2.7%。**KneadData 在右下角:**宿主清得最干净,却误删了近 1% 的微生物序列。**RustyClean auto 落在拐点**——两个方向都接近

最优,F1 因此最高。悬停任一标记可看具体数值。

正类为"微生物序列被保留"。FP 是没能清除的宿主序列(污染残留), FN 是被误删的微生物序列(数据损失)。两者不可同时最小化,四个工具正好排成一条单调的取舍曲线。

数据集	工具	宿主残留 FP	微生物误删 FN	F1
5M_1pct_low_even_SE	Hostile	1,023 • 2.05%	0 • 0.000%	0.9999
	RustyClean --skip-qc	859 • 1.72%	123 • 0.002%	0.9999
	RustyClean auto	209 • 0.42%	7,712 • 0.156%	0.9992
	KneadData	149 • 0.30%	43,739 • 0.884%	0.9955
10M_10pct_med_even_SE	Hostile	26,252 • 2.63%	0 • 0.000%	0.9985
	RustyClean --skip-qc	23,326 • 2.33%	596 • 0.007%	0.9987
	RustyClean auto	4,151 • 0.42%	5,269 • 0.059%	0.9995
	KneadData	3,077 • 0.31%	61,114 • 0.679%	0.9964
30M_50pct_high_skewed_SE	Hostile	399,819 • 2.67%	0 • 0.000%	0.9868
	RustyClean --skip-qc	310,060 • 2.07%	1,244 • 0.008%	0.9897
	RustyClean auto	39,892 • 0.27%	13,481 • 0.090%	0.9982
	KneadData	30,897 • 0.21%	95,774 • 0.638%	0.9958
60M_90pct_high_lognormal_SE	Hostile	1,057,928 • 1.96%	0 • 0.000%	0.9190
	RustyClean --skip-qc	889,123 • 1.65%	687 • 0.011%	0.9310
	RustyClean auto	135,380 • 0.25%	5,531 • 0.092%	0.9884
	KneadData	111,645 • 0.21%	58,174 • 0.970%	0.9859

四个数据集的中位数	宿主残留	微生物误删	F1	
Hostile		2.63%	0.000%	0.9985
RustyClean --skip-qc		2.07%	0.008%	0.9987
RustyClean auto		0.42%	0.092%	0.9992
KneadData		0.30%	0.884%	0.9958

三点值得写进论文:**RustyClean auto 的 F1 最高**(0.9992); KneadData 的宿主清除最彻底(0.30%),代价是误删的微生物序列比 RustyClean auto **多近十倍**(0.884% vs 0.092%), F1 因此垫底;**RustyClean --skip-qc 在两条轴上都优于 Hostile**——宿主残留低 0.56 个百分点,而多付出的微生物损失只有 0.008%。在 60M / 90% 宿主这个极端点上,Hostile 残留 105 万条宿主序列, RustyClean auto 只有 13.5 万条。

索引消融:结果与预期相反

v1 的推断是:混合库(`kraken16`)下 Kraken2 会把宿主序列归到人类的祖先节点,从而逃过按 taxid 的宿主判定,**因而应当有更高的宿主残留**。实测把这个推断推翻了。

数据集	宿主残留 混合库	人类专用库	微生物误删 混合库	人类专用库
30M_50pct_high_skewed_SE	0.079%	0.249%	0.084%	0.091%
60M_90pct_high_lognormal_SE	0.101%	0.238%	0.084%	0.094%
100M_50pct_high_lognormal_SE	0.054%	0.216%	0.085%	0.095%
100M_90pct_high_lognormal_SE	0.089%	0.279%	0.085%	0.096%
平均	0.081%	0.245%	0.084%	0.094%

混合库在两条轴上都更好:宿主残留低约 3 倍(0.081% vs 0.245%),微生物误删也略低, F1 为 0.99729 对 0.99320。一个合理的解释是:人类专用库里,任何与人类共享少量 k-mer 的序列 **都没有别的归属可选**;混合库给了 Kraken2 微生物的参照,反而判得更准。v1 担心的"LCA 逃逸"没有出现——至少在开启 recheck 的情况下没有。

这个结果**不支持**为祖先 taxid 补丁而合并 `feat/host-taxid-lineage` 分支: 它要解决的问题在实测中并不存在。两臂都开启了 recheck,但验证通道对 F1 的影响只有 0.0004 (见下),**不足以掩盖 3 倍的宿主残留差异**,所以这个结论是稳的。

auto 路由:估计准确,切换点符合设计

数据集	标称宿主比例	抽样估计	选择的后端
5M_1pct_low_even_SE		1%	0.98% bowtie2
5M_5pct_low_even_SE		5%	4.98% bowtie2
10M_10pct_med_even_SE		10%	9.87% bowtie2
30M_50pct_high_skewed_SE		50%	49.41% kraken2
30M_70pct_med_lognormal_SE		70%	69.23% kraken2
30M_90pct_med_lognormal_SE		90%	89.16% kraken2
60M_90pct_high_lognormal_SE		90%	89.68% kraken2
60M_99pct_med_lognormal_SE		99%	98.63% kraken2

十万条序列的抽样估计与标称值最大偏差 0.59 个百分点,八个数据集全部正确路由: $\leq 9.87\%$ 走 bowtie2, $\geq 49.41\%$ 走 kraken2。这条规则的两半——估计准确、切换合理——都得到了验证。

04 软件

被测工具是 RustyClean;KneadData 与 Hostile 是对照工具,均使用各自官方发布的默认数据库,不做修改。 其余为 RustyClean 各后端所依赖的比对/分类程序。

软件	角色	用于	位置
RustyClean	被测工具	stage 3-5 全部实验	rustyclean/target/release
KneadData	对照工具	stage 3 主对比	conda envs/kneaddata
Hostile	对照工具	stage 3 <code>--skip-qc</code> 对比	~/.local/bin
InSilicoSeq	序列模拟	stage 2 生成全部数据	.conda_envs/rustyclean-benchmark
Kraken2	RustyClean 后端	k-mer 分类去宿主	conda envs/kraken2
Bowtie2	RustyClean 后端	比对去宿主 + recheck 验证通道	conda envs/metaphlan
minimap2	RustyClean 后端	后端对比	.conda_envs/rustyclean-benchmark
sylph	RustyClean 后端	sketch 预筛	conda envs/sylph
centrifuge	RustyClean 后端	后端对比	.conda_envs/rustyclean-benchmark
fastp	质控	RustyClean 默认 QC 步骤	conda envs/metagem
samtools / seqtk	辅助	比对输出处理 / auto 模式抽样	conda envs

05 参考序列与数据库

核心设计:宿主序列从 GRCh38 模拟,去宿主对 T2T 进行。两者是不同的组装版本,因此去宿主必须应对真实的组装差异,而不是匹配序列的来源本身。 本项目构建的五个索引

全部来自同一份 T2T-CHM13v2.0 FASTA,并带 provenance stamp 记录来源, 因此后端之间的差异归因于算法,而非参考内容。

参考序列

用途	来源	说明
去宿主参考	T2T-CHM13v2.0 GCF_009914755.1	五个自建索引的共同来源
宿主序列的模拟来源	GRCh38.p14 GCF_000001405.40	刻意与去宿主参考不同
微生物群落取材	GTDB r202 参考基因组	9,057 个,按复杂度定种子随机抽样
分类信息	NCBI taxonomy	Kraken2 与 centrifuge 共用同一份

本项目构建的索引

索引	构建方式	体积	用于
Kraken2 human-only 默认后端	kraken2-build --build --kmer-len 35 --minimizer-len 31	hash.k2d 4.3 GB	除消融外的全部实验
Bowtie2 T2T-only	bowtie2-build(不加 --large-index)	.bt2 小索引	低宿主比例的比对后端、auto 抽样、recheck
minimap2	minimap2 -x sr -d	t2t_only.mmi	后端对比
sylph	sylph sketch -g	t2t.sylpdb	sketch 预筛
centrifuge	centrifuge-build,全部序列映射到 taxid 9606	.1.cf 999 MB .2.cf 372 MB	后端对比

Kraken2 库只含人类, `kraken2-inspect` 报告 **33 个分类节点**——正是人类谱系的深度。Bowtie2 索引原先用 `--large-index` 建成 `.bt21` ,该标志继承自 GRCh38+T2T 合并参考的脚本(约 6.2 Gbp,确实超过 Bowtie2 小索引 4 Gbp 的上限);T2T 单独 3.1

Gbp 并不需要, 且 `.bt21` 用 64 位偏移量、体积更大, 会让报告的索引占用相对 KneadData 与 Hostile 的 `.bt2` 被高估, 已重建。

消融对照库

库	内容	体积	角色
Kraken2 mixed kraken16	多类群库 , 73,690 个基因组, 人类只是其中之一	hash.k2d 15 GB names.dmp 258 MB nodes.dmp 194 MB	仅索引消融; 不是默认

「混合库」指的是多类群, 不是 **T2T + HLA**。config 中另有一个 `KRAKEN2_DB_T2T_HLA` 的定义, 但本系统没有 IPD-IMGT/HLA 序列, 该库**从未构建**, 任何实验都没有用到它。混合库的哈希表是人类专用库的 3.5 倍(15 GB vs 4.3 GB), 这也是消融臂需要把库暂存到节点本地磁盘的原因。

对照工具的官方库

工具	库	内容
KneadData	hg39_T2T	T2T-CHM13v2.0(GCF_009914755.1), 不含 HLA
Hostile	human-t2t-hla	T2T-CHM13v2.0 + IPD-IMGT/HLA

一处必须在论文中说明的不对称

Hostile 的默认索引在 T2T 之外还包含 IPD-IMGT/HLA, 而本系统没有 HLA 序列, 因此 RustyClean 与 KneadData 的索引都是纯 T2T。三方中只有 **Hostile 多了 HLA**。这个差异是比较本身的属性, 应当如实报告, 而不是通过改动 Hostile 的默认配置来抹平——使用者拿到的就是这个默认库。

=== 数据集 ===== -->

06 模拟数据集

18 个数据集, 合计 5.4 亿条序列, 覆盖测序深度(5M-100M)、宿主比例(0-100%)、群落复杂度(5/30/100 物种)、丰度分布(均匀/对数正态/偏斜)与测序模式(单端/双端)。群落按复杂度用固定种子随机抽样, 同一复杂度的数据集共用同一群落, 因此跨深度与跨宿主比例可比。

数据集	序列数	宿主	复杂度	丰度分布	模式	种子
5M_1pct_low_even_SE	5,000,000	1%	low • 5 种	even	SE	43
5M_5pct_low_even_SE	5,000,000	5%	low • 5 种	even	SE	44
10M_0pct_med_lognormal_SE	10,000,000	0%	med • 30 种	lognormal	SE	59
10M_1pct_med_lognormal_SE	10,000,000	1%	med • 30 种	lognormal	SE	45
10M_5pct_med_lognormal_SE	10,000,000	5%	med • 30 种	lognormal	SE	46
10M_10pct_med_even_SE	10,000,000	10%	med • 30 种	even	SE	47
10M_30pct_med_lognormal_SE	10,000,000	30%	med • 30 种	lognormal	SE	48
10M_100pct_med_lognormal_SE	10,000,000	100%	med • 30 种	lognormal	SE	60
20M_10pct_med_even_PE	20,000,000	10%	med • 30 种	even	PE	57
20M_50pct_med_lognormal_PE	20,000,000	50%	med • 30 种	lognormal	PE	49
20M_90pct_med_lognormal_PE	20,000,000	90%	med • 30 种	lognormal	PE	58
30M_50pct_high_skewed_SE	30,000,000	50%	high • 100 种	skewed	SE	50
30M_70pct_med_lognormal_SE	30,000,000	70%	med • 30 种	lognormal	SE	51
30M_90pct_med_lognormal_SE	30,000,000	90%	med • 30 种	lognormal	SE	52
60M_90pct_high_lognormal_SE	60,000,000	90%	high • 100 种	lognormal	SE	53
60M_99pct_med_lognormal_SE	60,000,000	99%	med • 30 种	lognormal	SE	54
100M_50pct_high_lognormal_SE	100,000,000	50%	high • 100 种	lognormal	SE	55
100M_90pct_high_lognormal_SE	100,000,000	90%	high • 100 种	lognormal	SE	56

模拟器为 InSilicoSeq, novaseq 误差模型,实测序列长度 151 bp。每个数据集写入 metadata.json 记录实测序列长度、模型与种子; 序列长度是从产出的序列中量取的,

而非写死的常数,因此 Methods 可以对照数据核实。

07 结论与待决

recheck 验证通道:建议不默认开启

两个方向都实现并测量过。旧版(额外删除)F1 +0.0004,代价 1.8 倍时间; 新版(挽回)方向正确、微生物误删确实下降,但**每挽回 1 条微生物序列要多留下 3.09 条宿主序列**, F1 反降到 0.99276,代价 7.2 倍时间。四个数据集合计净损失 6,594 条序列。

根本原因是可做的事太小:Kraken2 与 Bowtie2 在这批数据上的分歧本就只有千分之几, 任何基于二者不一致的复核都只能在这个量级内腾挪。 **建议不默认开启,并在论文中如实说明这个特性未能带来可测的收益**—— 这比略去它更有价值,因为它排除了一条看似合理的改进路径。

论述重心应从速度移到准确性

对 Hostile 的速度是持平(中位 0.99 $\times$),但准确性上 RustyClean `--skip-qc` 在两条轴上都更优。对 KneadData 则相反:速度快 4.4–8.6 倍,而 KneadData 的宿主清除 略胜一筹(0.30% vs 0.42%),代价是误删近十倍的微生物序列。 论文的主张应当是「在两类错误之间取得更好的平衡」,而不是单纯的速度。

三次重复只测量了时间波动

消融与 recheck 的 rep1/rep2/rep3 每个准确性数字都一模一样。这符合预期—— 确定性工具、同样输入、输出必然相同——但意味着这三次重复在准确性上没有提供额外信息, 论文中不应把它们当作独立重复。

索引消融的新版臂未完成,但不影响结论

混合库 + 新版 recheck 的 5 个任务中有 2 个因节点本地磁盘写满而失败: 新版把 5400 万条宿主序列的比对结果写成了约 21 GB 的 SAM,与 16 GB 的暂存库挤在同一块 `/tmp` 上。已改为流式读取、不再落盘。

不必为此重跑消融:消融要回答的是索引问题,而现有的旧版消融结果已经给出了答案 (混合库宿主残留低约 3 倍),且 recheck 对 F1 的影响不超过 0.0004,不足以改变那个结论。

决策边界臂的超时:数据完整,无需重跑

`rc_auto_boundary` 有一次尝试在 6 小时时限处超时,但该臂的六个数据集 **各有完整且正常的测量**(10M/30% 实测 bowtie2 612 秒、kraken2 137 秒),说明超时的是一次被取代的尝试,很可能输给了节点争用——同期有多个 6–18 小时的作业在压同一文件系统。时限已提高到 24 小时与其他臂一致,把 walltime 从变量里去掉。